1. The Request Lifecycle

The journey of an HTTP request is a series of layers, moving from the network level into the Spring Context.

1. **Client Request:** A client sends an HTTP request (e.g., POST /api/logs).
2. **Web Server (Tomcat):** The request hits the embedded server.
3. **Filters:** The request passes through the **Servlet Filter** chain (Security, Logging).
4. **DispatcherServlet:** The "Front Controller" receives the request and begins the Spring-specific processing.
5. **Handler Mapping:** DispatcherServlet asks HandlerMapping to find the right @Controller.
6. **Interceptors:** Before the controller is hit, preHandle logic is executed.
7. **Controller Action:** Your code in the @RestController runs.
8. **Response Return:** The result flows back through Interceptors (postHandle), then through the Filters, and finally back to the client.

---

2. DispatcherServlet: The "Front Controller"

The DispatcherServlet is the heart of Spring MVC. It acts as a central coordinator.

- **What it does:** Instead of having 50 different Servlets for 50 different URLs, Spring uses one DispatcherServlet.
- **How it works:** It delegates tasks to specialized components:
    - **HandlerMapping:** To find the controller.
    - **HandlerAdapter:** To actually execute the controller method.
    - **ViewResolver:** (In traditional MVC) To find the HTML page. In a @RestController, this is bypassed in favor of an **HttpMessageConverter** (like Jackson for JSON).

---

3. @RestController Flow

A @RestController is a convenience annotation that combines @Controller and @ResponseBody.

- **The Serialization Magic:** When your method returns a Java object (e.g., a LogBatch), the DispatcherServlet looks for an HttpMessageConverter.
- **Process:** 1. The return value is intercepted.

2. Spring finds the **Jackson** library in the classpath.

3. Jackson serializes the Java Object into **JSON**.

4. The JSON is written directly to the HTTP Response body with a 200 OK status.

---

4. Filters vs. Interceptors

This is a classic interview question. Both are used for "cross-cutting concerns," but they live in different worlds.

Feature	**Filter (Servlet API)**	**Interceptor (Spring Context)**
Origin	Part of the Web Container (Tomcat).	Part of the Spring Framework.
Awareness	Knows about the HTTP Request/Response only.	Knows about Spring Beans and which Controller/Method is being called.
Execution	Executes **outside** the DispatcherServlet.	Executes **inside** the DispatcherServlet.
Use Case	Low-level tasks: Security (Spring Security), GZIP compression, Logging IPs.	High-level tasks: Checking User Permissions for a specific method, adding common model attributes.

---

5. Practical Scenario: The "Audit Log"

The Interviewer asks: "Where would you put code that logs the execution time of every API call?"

The Senior Answer:

"I would use a **HandlerInterceptor**. Since it has access to the Handler (the controller method), I can use preHandle to record the start time and afterCompletion to calculate the total duration. If I needed to block a request based on a malformed header before it even reached Spring, I would use a **Filter**."

Final Technical Summary Checklist

- **Request Lifecycle:** Can you trace a request from Tomcat to a Controller?
- **DispatcherServlet:** Do you understand it's a central dispatcher, not the business logic handler?
- **Serialization:** Do you know that Jackson handles the POJO-to-JSON conversion?
- **Scopes:** Do you know that most Controllers are **Singletons** and must be thread-safe?